Heuristuques & Méta-Heuristiques

Hala Bezine

M. de Conférences

Faculté des Sciences de Sfax- Département informatique & Communications

PLAN

- Introduction
- Optimisation & la prise de décision
- Optimisation Combinatoire
- Méthodes de résolution Approchées
- Algorithmes à solution unique / génération de solutions

L'optimisation, un enjeu omniprésent

- Chaque processus a un potentiel d'être optimisé.
- Toute entreprise est souvent confrontée à résoudre des problèmes d'optimisation.
- Les applications de l'optimisation dans la vie courante sont innombrables.
- En effet, de nombreuses applications difficiles de la science et de l'industrie peuvent être formulées comme des problèmes d'optimisation.
- L'optimisation se produit dans la minimisation du temps, du coût et du risque, ou la maximisation du profit, de la qualité et de l'efficacité, etc.

L'optimisation, un enjeu omniprésent

- Optimisation
  - Définition du comportement à atteindre
  - Calcul des paramètres du système pour atteindre

- Solution
  - Un ensemble de paramètres du système permettant d 'obtenir (+/-) le comportement souhaité

Exemples

1

Réseaux

Concevoir un réseau afin d'optimiser le coût et la qualité du service.

2

Production

Planifier une production pour optimiser le temps et le profit économique.

3

Bio-informatique

Prédire la structure 3D d'une protéine afin d'optimiser l'énergie potentielle.

4

Transport

Planifier les marches de trains afin d'optimiser le temps et la qualité de service.

… et bien d'autres domaines encore.

Étapes d'un processus de décision

1

Formuler

Identifier et décrire le problème de décision.

2

Modéliser

Construire un modèle mathématique abstrait.

3

Optimiser

Résoudre le modèle et produire une solution.

4

Implémenter

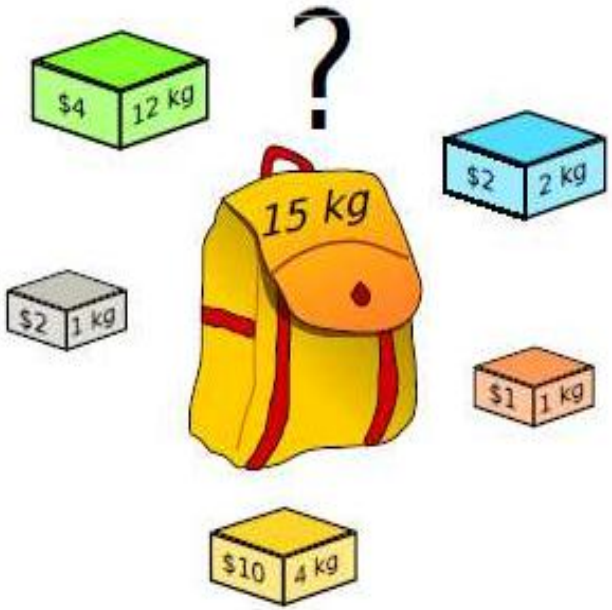
Tester et mettre en œuvre la solution.

Problème d'optimization combinatoire ?

- Un problème d'optimisation combinatoire est un problème d'optimisation dans lequel l'espace de recherche S est dénombrable.
- Sa résolution consiste à rechercher une solution optimale (optimisation) dans un espace comportant un très grand nombre de solutions possibles (combinatoire).
- Un problème d'optimisation combinatoire peut être :
  - ***sans contraintes*** *: on cherche une solution qui optimise la fonction objectif.*
  - ***avec contraintes*** *: on cherche une solution qui optimise la fonction objectif et qui vérifie les contraintes du problème.*

Exemple 1; Problème du Sac à Dos

- Heuristique possible: rajouter en priorité les objets ayant le meilleur rapport valeur/poids, jusqu'à ce que le sac soit rempli



Exemple 1; Problème du Sac à Dos

Modélisation par PL programmation Linéaire

- On introduit N variables de décision : $x_i \in \{0, 1\}$
- $x_i = 0$ (resp
- N contrair
- Contrainte
- Fonction o

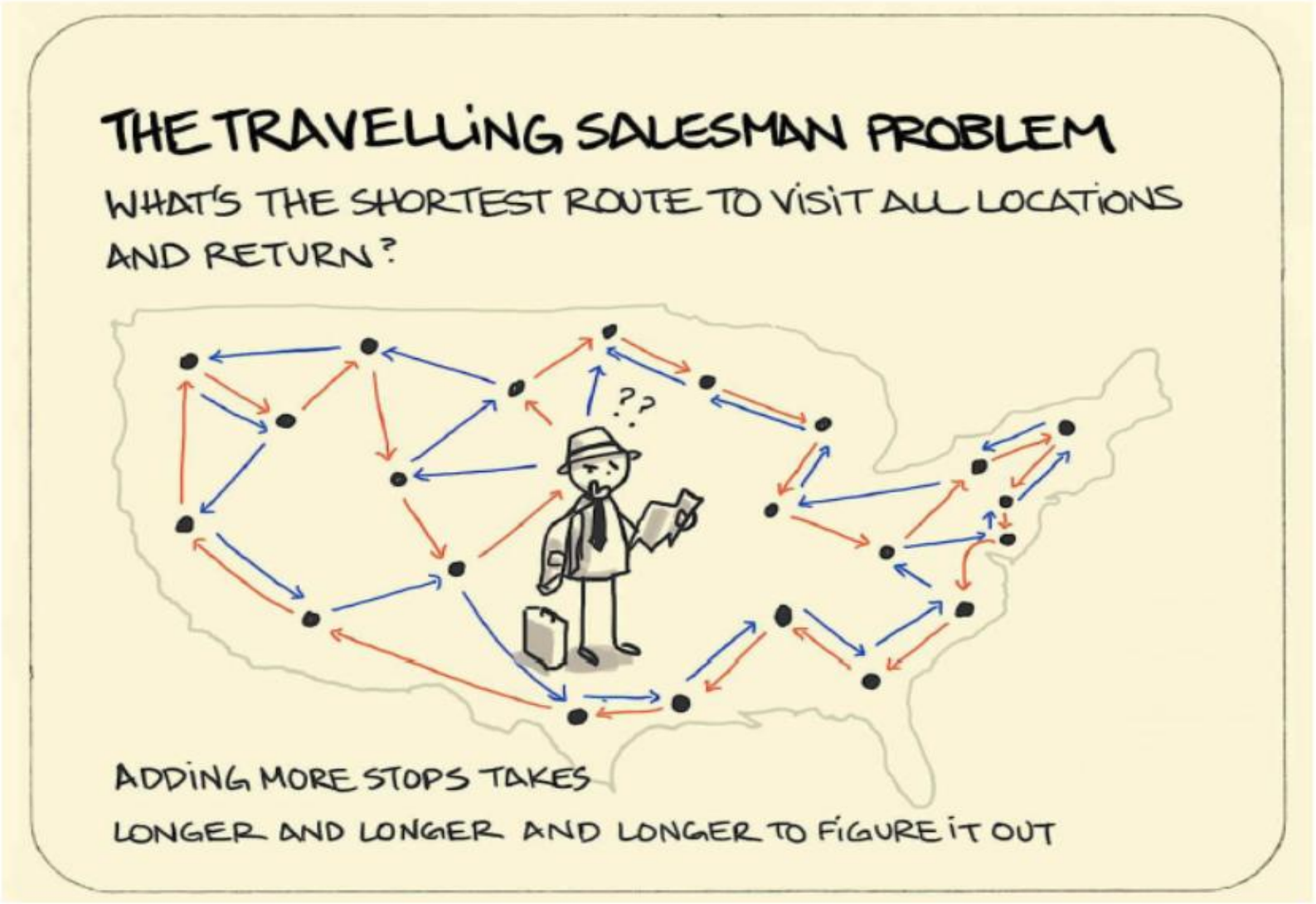
$$\text{Max } z = \sum_{i=1}^{n} x_i u_i$$

$$\text{s.c}$$

$$\sum_{i=1}^{n} x_i p_i \leq C$$

$$x_i \in \{0, 1\}$$

Exemple 2; Problème du Voyageur de Commerce



Exemple 2; Problème du Voyageur de Commerce (TSP)

INSTANCE

1. Un représentant et N clients, chacun dans une ville (N villes).
2. La matrice des distances entre les villes : d_{ij} distance entre les villes i et j.

QUESTION

- Trouver un ordre de visite de toutes les villes afin que le représentant parcoure une distance totale minimale, tout en passant au moins une fois par chacune des villes.

Modélisation du TSP en termes de graphe

- Le TSP se représente par un graphe G = (V, E) dont les sommets (les éléments de V) sont les villes et dont les arêtes (les éléments de E) représentent les routes reliant les villes.
- Le problème consiste à déterminer l'itinéraire optimal, ce qui revient alors à trouver dans le graphe G = (V, E) un cycle de longueur minimale passant au moins une fois par chaque sommet (cycle hamiltonien minimal).

FORMULATION ÉQUIVALENTE

Chercher un cycle hamiltonien de poids minimal dans un graphe pondéré complet.

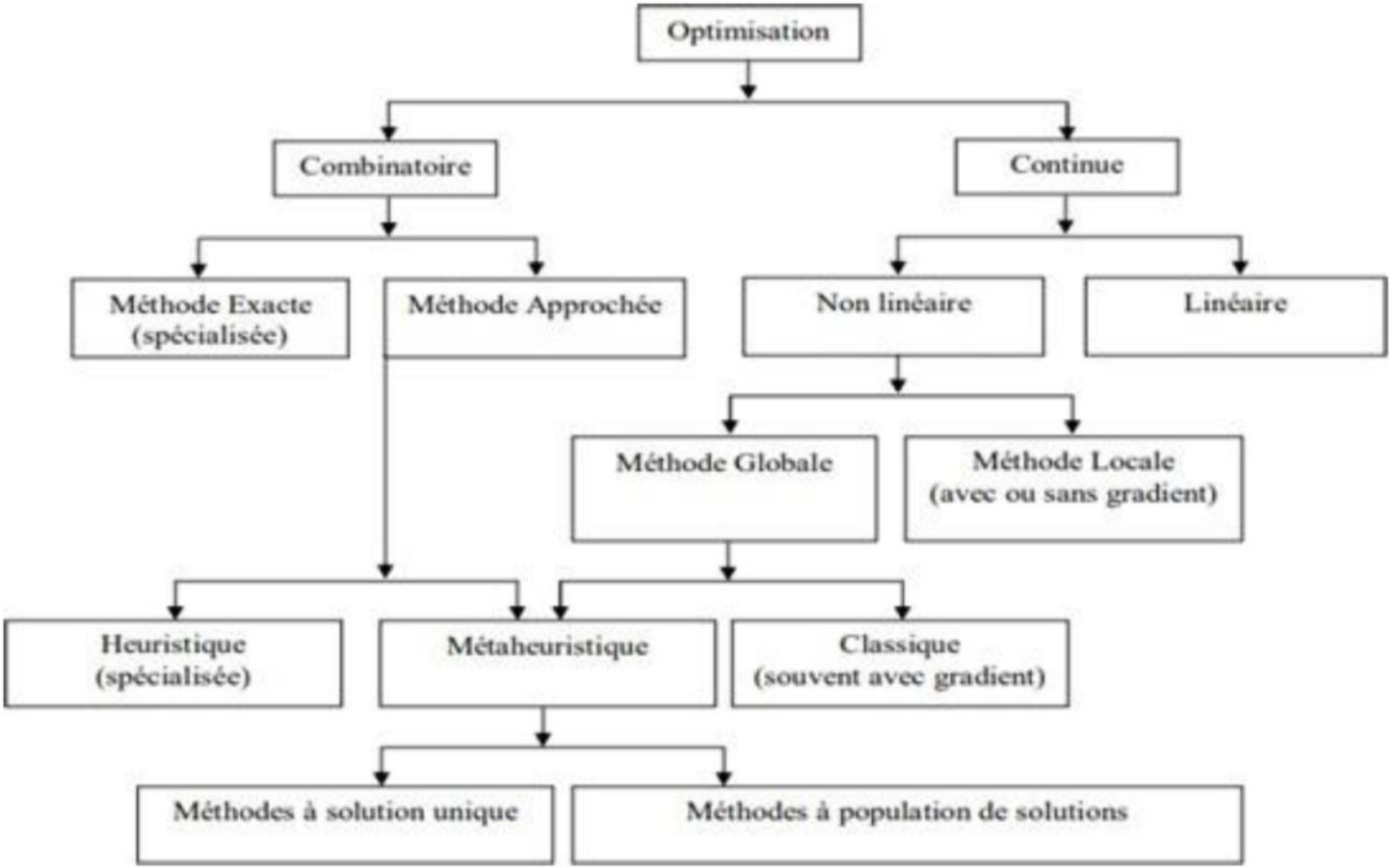
Résolution exacte du TSP

- Le TSP est très facile à modéliser.
- Mais il est très difficile à résoudre, puisque le nombre de possibilités est de l'ordre de N ! = O(N !).
- On parle d'explosion combinatoire.

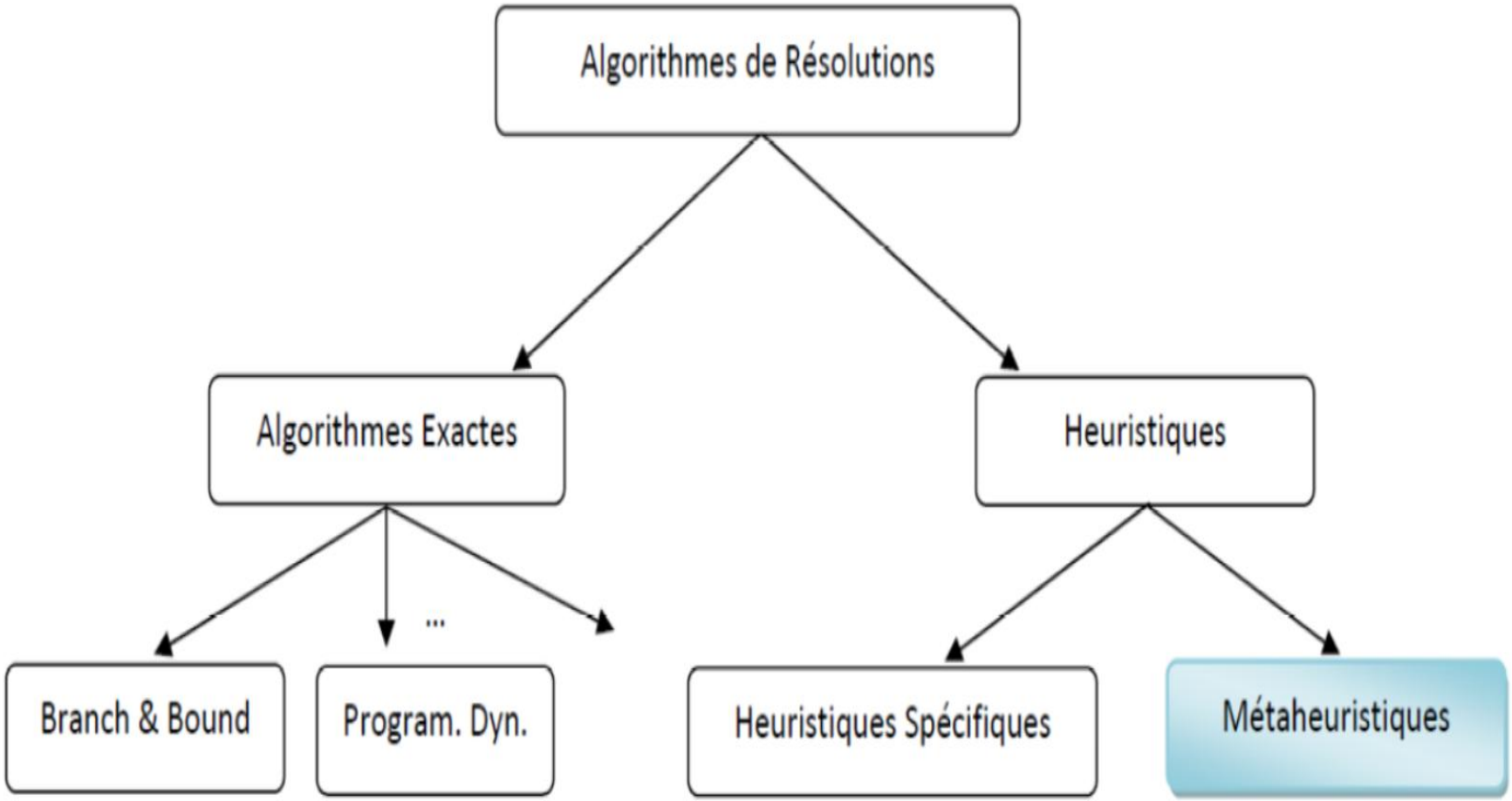
O(N !)

Nombre d'itinéraires à explorer

Optimization & Méthodes de résolution



Optimization combinatoire & Méthodes de résolution



Algorithme Exact Vs Algorithme Approché

- Algorithme exact : garantit une solution optimale
- Algorithme approché : pas de garantie d'optimalité
- Les méthodes exactes ne sont efficaces que pour les instances de problèmes de petite taille
- Les problèmes réels sont en général de (très) grande taille
- Les méthodes approchées sont plus efficaces pour les problèmes de grande taille
- On ne s'intéressera ici qu'aux méthodes approchées

Approches de resolution des POC

1

Méthodes exactes

Trouvent toujours une solution optimale du problème.

2

Méthodes approchées sans garantie

Heuristiques et métaheuristiques, sans garantie de performance.

Méthodes exactes

- ***Une méthode exacte trouve toujours une solution optimale d'un POC.***
- Il y a plusieurs méthodes de résolution exactes, parmi lesquelles :

1. Les méthodes de type Branch-and-X (X = Bound, Cut, Price).
2. Les méthodes par retour en arrière (Backtracking).
3. Les algorithmes gloutons.
4. La programmation dynamique.
5. La programmation linéaire.

Notion d'heuristique

- **Une heuristique** : du grec ancien eurisko, « trouver »
- C'est un moyen (un critère, une procédure, un ensemble de règles) destiné à réduire les alternatives et guider les choix non déterministes que doit faire un algorithme de recherche
- indispensable pour les problèmes NP-difficiles car généralement en temps polynomial
- Une heuristique exploite généralement une information spécifique au problème posé
- traduit une stratégie, une manière de penser, s'appuyant sur notre connaissance du problème

Heuristiques; définitions

Heuristique : algorithme de résolution basé sur l"'expérience" ne fournissant pas nécessairement une solution optimale

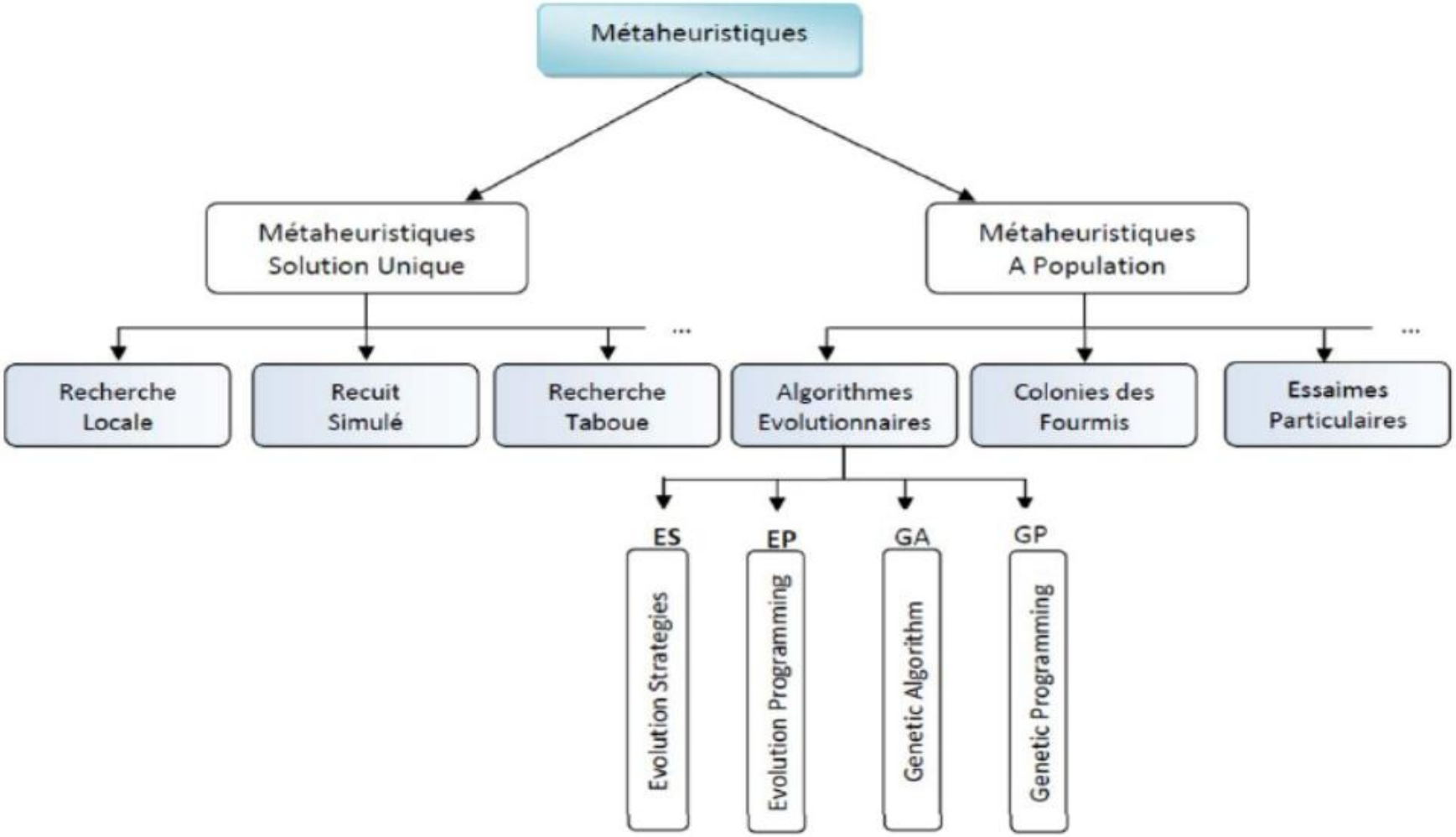
On désire toutefois :

- Le plus souvent possible une solution proche de l'optimalité
- Le moins souvent possible un mauvaise solution (différent !)
- Une complexité "raisonnable"
- De la simplicité d'implémentation (code light en version de base...)

Notion d'Heuristique

- **Une heuristique** : du grec ancien eurisko, « trouver »
- C'est un moyen (un critère, une procédure, un ensemble de règles) destiné à réduire les alternatives et guider les choix non déterministes que doit faire un algorithme de recherche
- indispensable pour les problèmes NP-difficiles car généralement en temps polynomial
- Une heuristique exploite généralement une information spécifique au problème posé
- traduit une stratégie, une manière de penser, s'appuyant sur notre connaissance du problème

Méthodes de résolution; Méta-Heuristiques



Méta-Heuristiques, Caractéristiques

Métaheuristique : ensemble d'heuristiques
Peu probable qu'un algorithme puisque résoudre tout problème

Métaheuristique :

- regroupe des heuristiques dépendant de paramètres
- décrive une méthode de conception d'heuristique

de
un aveu d'impuissance
à
des techniques performantes d'optimisation difficile

Heuristiques

Heuristique : algorithme de résolution basé sur l'"'expérience" ne fournissant pas nécessairement une solution optimale

On désire toutefois :

- Le plus souvent possible une solution proche de l'optimalité
- Le moins souvent possible un mauvaise solution (différent !)
- Une complexité "raisonnable"
- De la simplicité d'implémentation (code light en version de base...)

Notion d'Heuristique

Résumé

- Il s'agit d'un moyen de choisir, parmi plusieurs directions, celle qui semble être la plus indiquée pour résoudre un problème
- Ce moyen peut être très efficace mais quelque fois peut induire en erreur et orienter vers une fausse piste.
- L'heuristique est généralement fondée sur une **simplification** du problème initial, une **relaxation des contraintes** qui font que ce problème est difficile
- La qualité d'une heuristique peut se mesurer en terme de réduction de la complexité de la recherche: une heuristique h1 est préférable à h2 si dans tous les cas h1 conduit à explorer moins d'alternatives que h2

Heuristique & Métaheuristique

- Méthodes heuristiques ou approchées (1)
  - Recherchent, à **moindre coût, une solution dont il n'est pas possible de garantir la qualité**
  - Une méthode heuristique est dite «robuste» si elle converge le plus souvent vers la même solution
  - Une méthode heuristique est dite «efficace» si, à temps de calcul donné, elle donne une solution proche de l'optimum
  - Parmi ces méthodes, on distingue
    - Les méthodes développées pour des problèmes spécifiques
    - Les méthodes adaptables à un grand nombre de problèmes différents sans changements majeurs dans l'algorithme

    On parle alors de méthodes «méta-heuristiques»

Pourquoi Les Métaheuristiques

- une heuristique est spécifique à un problème et ne peut pas être généralisée
- méta + heuristique = au-delà + trouver ➔ trouver avec un plus haut niveau d'abstraction
- une métaheuristique est un ensemble de concepts : voisinage (modification d'une solution), utilisation de la mémoire...
- une méta-heuristique est une heuristique généraliste, pouvant s'appliquer à plusieurs problèmes d'optimisation

INTRODUCTION

- Les ***métaheuristiques*** utilisent deux approches pour résoudre tel problème d'optimisation,

- **L'approche basée population;** elle désigne les algorithmes qui traitent plusieurs solutions à la fois. Elles maintiennent et améliorent plusieurs solutions candidates en même temps. On retrouve les Algorithmes Génétiques, Les algorithmes de fourmis, les algorithmes d'optimisation par Essaims de particules.

- **L'approche à solution Unique:** elle représente les algorithmes qui font évoluer une fonction objective unique à chaque itération. La stratégie est basée sur une recherche locale (Recuit simulé, etc...

INTRODUCTION (méthodes incomplètes

- Constat : pour de nombreux problèmes (taille), aucune technique vue precédemment n'est efficace
- Les techniques étudiées sont complètes (ou exactes) : si elles ne fournissent pas de solution, la solution n'existe pas
- Idée : peut-on trouver des méthodes incomplètes qui soient efficaces dans certains cas ?
- Parmis ces méthodes, on trouve les méta-heuristiques

Pourquoi Les Métaheuristiques

Alors ***une métaheuristique*** est une méthode algorithmique capable de guider et d'orienter le processus de recherche dans un espace de solution, souvent très grand à des régions riches en solutions optimales. A ces applications, les métaheuristiques permettent, de trouver des solutions, peut être pas toujours optimales, en tout cas très proches de l'optimum et en un temps raisonnable. Elles **se distinguent** en cela des **méthodes dites exactes**, qui garantissent certes la résolution d'un problème, mais au prix de temps de calcul prohibitifs.

- L' existence de minima locaux, impose l'utilisation de méthodes d'exploration efficaces pour éviter de rester bloqué aux alentours de ces minima : mécanismes qui assurent le compromis Intensification/Diversification
- Plusieurs métaheuristiques basées sur la recherche locale ont été proposées :
  - *Recuit simulé* est basé sur les principes d'équilibre énergétique lors de la cristallisation des métaux ;
  - *Recherche tabou* introduit la notion d'histoire (mémoire) dans la stratégie d'exploration des solutions.

Approches de résolution des POC

Les heuristiques sont aussi très utiles pour le développement de méthodes exactes fondées sur des techniques d'évaluation et de séparation (Branch and Bound).

Métaheuristiques 1.16

- *Voici les métaheuristiques que l'on va étudier dans le cadre de ce cours :*
  1. *Recherche locale (Local Search).*
  2. *Recherche taboue (Tabu Search).*
  3. *Recuit simulé (Simulated Annealing).*
  4. *Les algorithmes génétiques (Genetic Algorithm).*
  5. *Algorithme d'optimisation par colonies de fourmis (Ant Colonies).*

Métaheuristique de Recherche Locale

Définitions:

Métaheuristique : ensemble d'heuristiques
Peu probable qu'un algorithme puisque résoudre tout problème

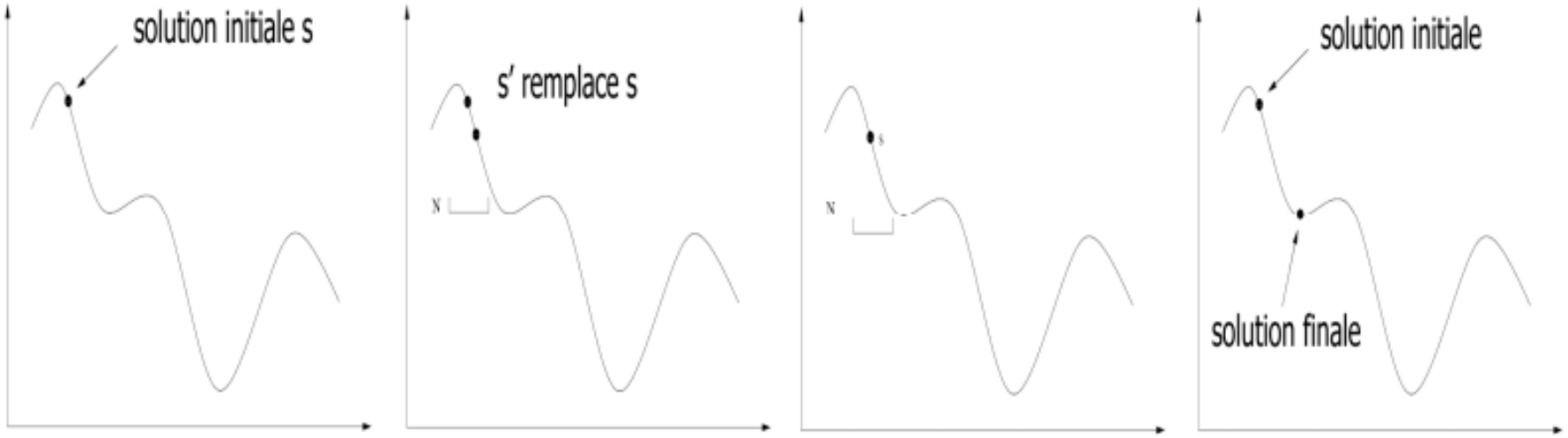
Métaheuristique :

- regroupe des heuristiques dépendant de paramètres
- décrive une méthode de conception d'heuristique

de
un aveu d'impuissance
à
des techniques performantes d'optimisation difficile

Descente de Gradient

Procédure *Gradient* (solution initiale *s*)
Répéter
Choisir *s'* dans V (s)
Si $f(s') < f(s)$ alors $s \leftarrow s'$
Jusqu'à ce que $f(s') \geq f(s), \forall s' \in S$
Fin



Variante: Méthode de la plus Grande Descente de Gradient

Procédure *Gradient* (solution initiale s)

Répéter

Choisir s' dans $V(s)$ tq $\forall\, s'' \in V(s)\ f(s') < f(s'')$

Si $f(s') < f(s)$ alors $s \leftarrow s'$

Jusqu'à ce que $f(s') \geq f(s), \forall\, s' \in S$

Fin

Descente de Gradient; Avantages & Inconvénients

- Principal avantage : simplicité et rapidité
- MAIS s'arrête au premier optimum local trouvé
- Améliorations possible:
  - Lancer plusieurs fois l'algorithme en partant d'un jeu de solutions initiales différentes
  - Autoriser de temps à autre une certaine dégradation des solutions trouvées afin de mieux explorer tout l'espace des configurations : conduit au développement de meilleures méthodes

Récuit simulé

- Dans un premier temps, *T* étant généralement choisi très grand ➔ beaucoup de solutions sont acceptées : Favoriser la diversification

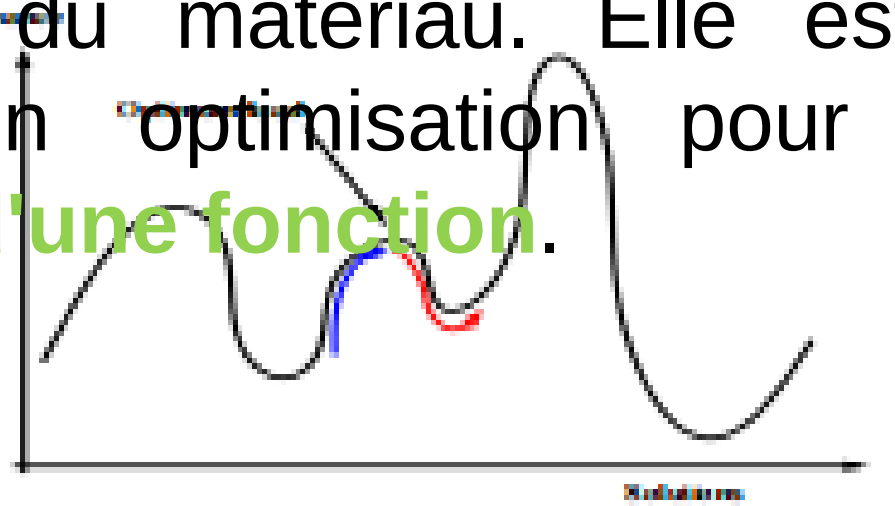
- A température intermédiaire, l'algorithme autorise de temps en temps des transformations qui dégradent la fonction objectif. Il laisse ainsi une chance au système de s'extraire d'un minimum local.

- À mesure que la température baisse, la plupart des mouvements augmentant l'énergie sont refusés : Intensification

Récuit Simulé

Système physique	Problème d'optimisation
Energie	fonction objectif
Etats du système	solution
Etats de basse énergie	bonne solution
Température	paramètre de contrôle

Le recuit simulé (Simulated Annealing en anglais) est une **métaheuristique** inspirée d'un processus utilisé en métallurgie. Ce processus alterne des cycles de refroidissement lent et de réchauffage (recuit) qui tendent à **minimiser l'énergie** du matériau. Elle est aujourd'hui utilisée en optimisation pour trouver les **extrema d'une fonction**.



Récuit Simulé

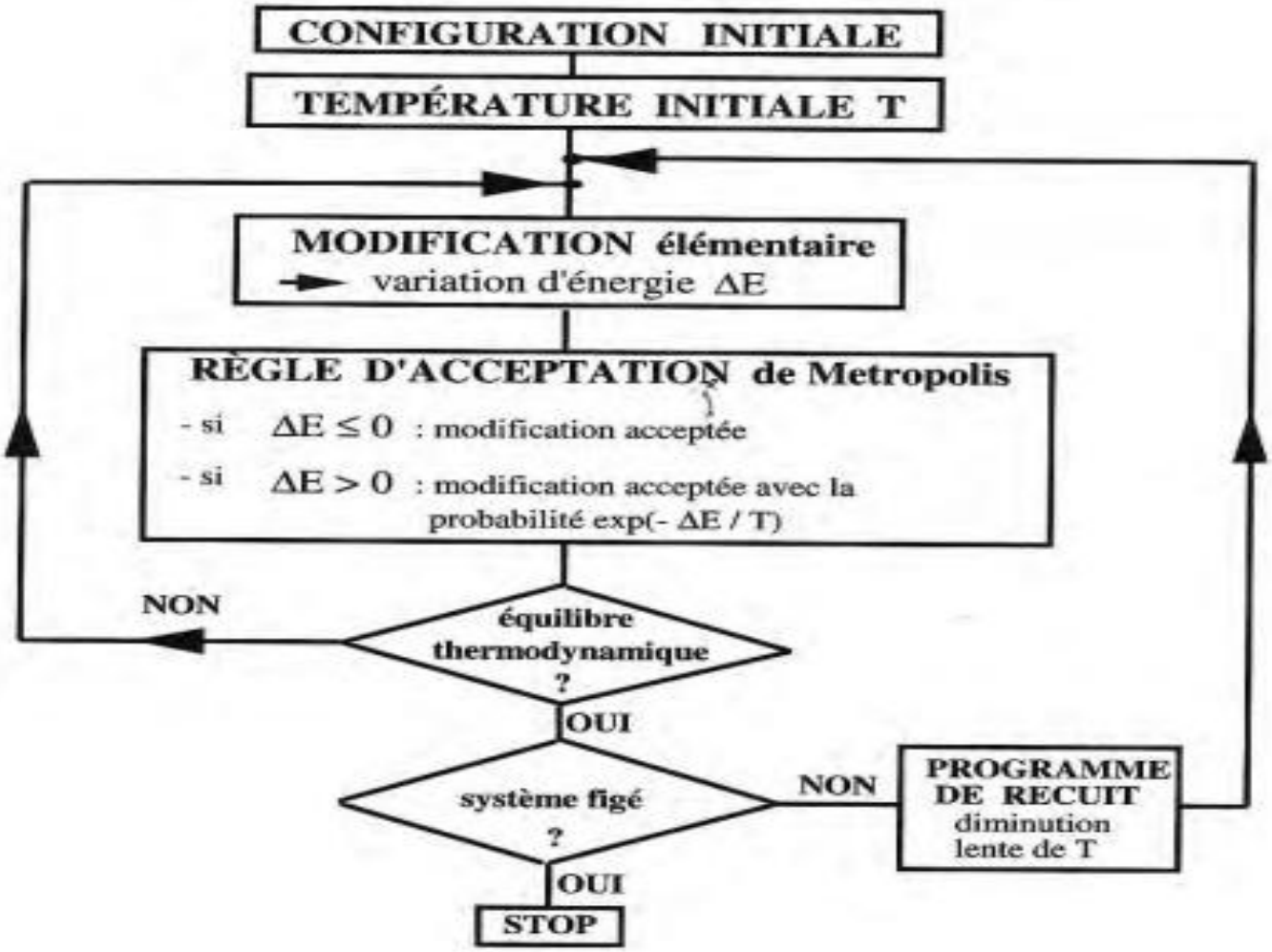
- Le recuit simulé est une méthode inspirée de la metallurgie
- Energie E du système = Fonction à minimiser
- Temperature T = paramètre contrôlant la diversification
- Determine conditions initiales : E_0 et T_0 (aléatoirement élevée)
- A chaque étape :
  - ➥ Modifie solution courante et évalue la variation d'énergie Δ_E
    - Si $\Delta_E < 0$, elle est appliquée
    - Sinon, elle est appliquée avec une probabilité de $e^{-\frac{\Delta_E}{T}}$ (règle de Metropolis)
  - ➥ Modification de la température : par paliers ou par une loi de décroissance ($T_{i+1} = \lambda T_i$ avec $\lambda < 1$)

Récuit Simulé

Algorithme 9 Recuit Simulé

1. $t \leftarrow t_0$; t_0 est la température initialiseée à une grande valeur
2. $s \leftarrow s_0$; s_0 une solution initiale
3. $best \leftarrow s$;
4. ***Tantque*** ($Best$ est la meilleure solution) & ($Nombre\ Iter. = \max$) & ($t \leq 0$) ***Faire***
5. $r \leftarrow s'$ où $s' \in N(s)$;
6. $\rho \leftarrow random(o, 1)$;
7. ***Si*** $\left(f(r) < f(s)\right)$ ***ou*** $\left(\rho < e^{\frac{f(r)-f(s)}{t}}\right)$ ***Alors***
8. $s \leftarrow r$;
9. ***Si*** $\left(f(s) < f(best)\right)$ ***Alors***
10. $best \leftarrow s$;
11. $décémenter\ t$;
12. ***Fin Tantque***

Récuit Simulé; algorithme



Recherche par Tabou

- La méthode tabou est une **méta-heuristique d'optimisation**.

- La méthode taboue consiste, à partir **d'une position donnée, à explorer le voisinage et à choisir** la position dans ce voisinage **qui minimise la fonction objectif**

1. Générer une position de départ.
2. Parmi les positions voisines, choisir la meilleure qui n'est pas dans la liste des tabous.
3. Si la liste des tabous est pleine, retirer de cette liste la position la plus ancienne.
4. Mettre la position actuelle en queue de la liste des tabous.
5. Retourner à 2, tant qu'on n'a pas décidé de s'arrêter.

Recherche par Tabou

Introduite par Glover en 1986 :
«Future paths for Integer Programming and Links to Artificial Intelligence», Computers and Operations Research, 5 :533-549, 1986.

But : échapper aux optima locaux
Principe : Introduction d'une notion de mémoire dans la stratégie d'exploration

Interdiction de reprendre des solutions déjà (ou récemment) rencontrées

Recherche par Tabou

Choisir solution initiale $s \in \mathcal{S}$
Initialiser Tabou M
répéter
  choisir $s' \in \mathcal{V}(s)$ telle que :
  ($f(s')$ meilleure solution de $\mathcal{V}(s)$ ET Critère d'aspiration vérifié)
  OU $f(s')$ meilleure solution de $\mathcal{V}(s)$ non taboue
  $s \leftarrow s'$
  update Tabou M
jusqu'à Critère d'arrêt vérifié

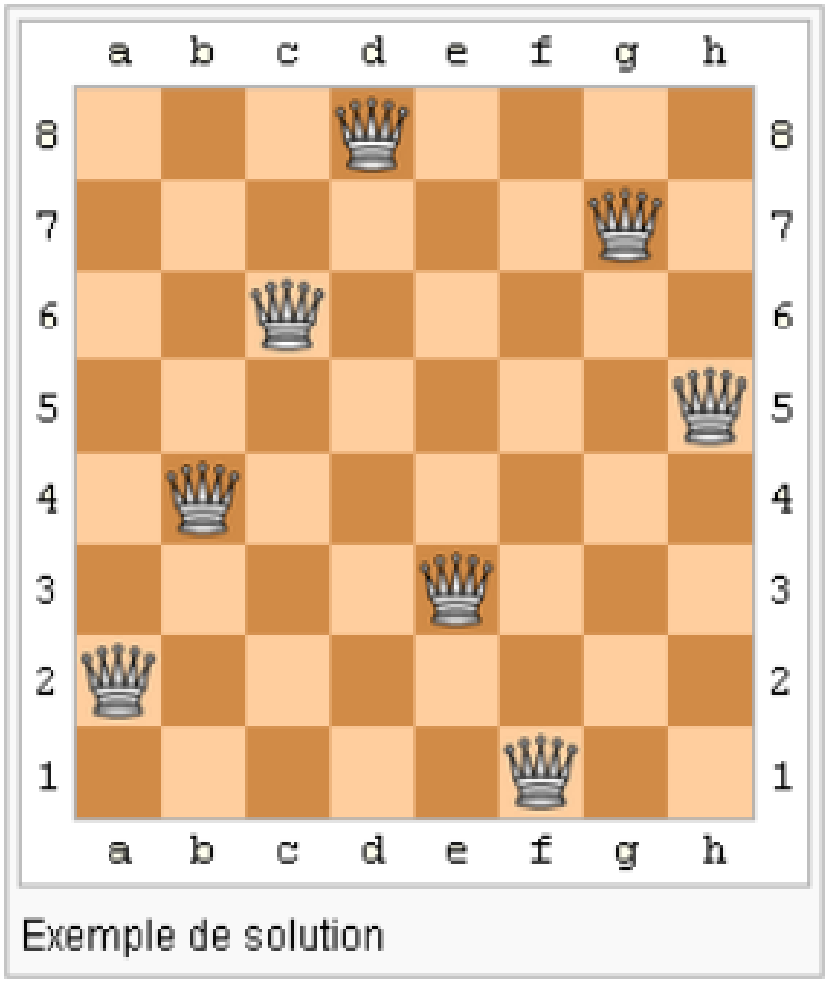
- Les **positions déjà explorées** sont conservées dans **une file; appellée liste de Tabou.**

Approches Constructives

Algorithmes gloutons (greedy algorithm)

- Principe : partir d'une solution initiale vide et ajouter, d'une manière incrémentale, des composants de solutions sans remettre en cause les choix antérieurs jusqu'à obtenir une solution complète.
- Deux questions essentielles :
  - Définir l'ensemble des composants (les éléments)
  - Comment sélectionner les composants qui donnent le profit optimal à chaque itération

Exemple;Pb de l'échequier(1/3); 8 reines



Exemple de solution

Un algorithme de « réparation itérative » commence typiquement à partir d'un placement de toutes les dames sur l'échiquier, par exemple avec **une seule dame par ligne et par colonne**. Il essaie ensuite toutes les permutations des colonnes (il n'y a que des conflits diagonaux à tester) pour **ne garder que les configurations sans conflits**.

Il y a 8! = 40'320 placements à explorer.

Exemple;Pb de l'échequier (2/3) 8 reines

◆ **Problème des 8 reines**

Trouver une configuration de 8 reines sans prise sur un échiquier. Aucune ligne, colonne, ou diagonale ne doit comporter plus d 'une reine.

Approche par transformation de solution potentielle

Partir d 'une configuration, la tester, si elle ne constitue pas une solution, la modifier localement, et recommencer.

Approche constructive

Procéder à chaque étape en plaçant une k^e reine sans prise sur un échiquier où figurent k-1 reines.

Exemple;Pb de l'échequier(3/3) 8 reines

Placez initialement les 8 dames sur la diagonale de l'échiquier, puis échangez deux colonnes choisies au hasard. **Répétez l'opération** jusqu'à ce **qu'une solution soit trouvée**.

on **peut coder la position** des dames très simplement : pour chaque colonne, on note sur quelle ligne se trouve la dame, et on lui soustrait 1. La position ci-contre sera : **[1, 3, 5, 7, 2, 0, 6, 4]**

- on passe très facilement d'une configuration à une configuration voisine (qui ne satisfait pas forcément les contraintes du problème) : il **suffit d'échanger deux colonnes.** Une position voisine de celle ci-contre pourrait être **[1, 0, 5, 7, 2, 3, 6, 4].**

Espace de Recherche

L'espace de recherche est une collection de solutions potentielles à un problème

- Comment chercher de bonnes solutions dans cet espace ?
- Il est important de connaitre les propriétés de cet espace, e.g.,:
  - Taille : liée à la représentation que l'on se fait des solutions
  - Structure : liée à la façon avec laquelle on se ballade/cherche dans cet espace (paysage de recherche)

Espace de Recherche

Taille de l'espace de recherche (SAT)

- Pour n=100 variables, la taille de l'espace de recherche est : 2^{100}
- Avec 1000 solutions évaluées / sec : il faudrait 15 billions d'années pour évaluer moins de 1% de l'espace de recherche
- Pour k>2, k-SAT est NP-difficile ; k=2, Polynomial